PROJECT REVIEW REPORT

AURA

Autonomous University Resource Allocator

Green Cloud Resource Optimization for Smart University Data Centers

BCSE355L – Cloud Architecture Design

Academic Year 2026–2027

Submitted by

Reg. No.	Name	Role
24BIT0539	Agrani Anupam	AI / Machine Learning
24BIT0548	Prakul Jain	Backend and Database
24BIT0537	Pranav Hasban	Frontend

Under the guidance of

Dr. Priya V

October 2026

Table of Contents

Abstract 3
1. Introduction 4
2. Problem Statement and Objectives 4
3. Related Work and Project Contribution 5
4. System Architecture 5
5. Dataset and Preprocessing 6
6. Demand Forecasting Model 7
7. Forecast-Driven, Carbon-Aware Allocation Simulation 8
8. Application Implementation 9
9. AWS Deployment Design 13
10. Testing and Quality Assurance 14
11. Results and Discussion 15
12. Limitations and Future Work 16
13. Conclusion 17
Acknowledgements 17
References 18
Appendix A. Running the Project 19
Appendix B. Configuration Reference 19
Appendix C. Glossary 20

Abstract

AURA cut simulated data-center energy use by 39.5% and CO_2 emissions by 41.3% against always-on provisioning, while meeting demand in 99.53% of hours. University data centers usually keep every server running all day, even though campus demand follows a predictable academic rhythm of lectures, weekends, holidays and exams. We built AURA (Autonomous University Resource Allocator) to use that rhythm. A two-layer LSTM forecasts the next 24 hours of load from the past 7 days, with calendar and Thai public-holiday features. A scheduler then keeps only the servers the forecast needs switched on, and moves deferrable batch work into the hours when the electricity grid is cleanest.

We trained and tested the system on real metered data from the CU-BEMS building at Chulalongkorn University in Bangkok, using a strict train/validation/test split. On the held-out test period (12 August to 21 October 2019), the LSTM's mean absolute error (MAE) was 35.9 kW, 20% lower than a same-hour-last-week baseline. AURA is a full-stack application: a React dashboard, a Node.js/Express API, and a Python forecasting engine. It is deployed on AWS using EC2, S3, Cognito, DynamoDB and IAM, and the dashboard includes an interactive simulator, CSV upload, anomaly detection and carbon advice.

1. Introduction

Data centers use about 1–1.5% of the world's electricity, and much of that is wasted on servers that sit idle. University data centers are a clear case. They host learning-management systems, student portals, research jobs and campus IoT, but they are usually sized for the busiest exam week and left running at that size all year.

Campus demand is not random, though. It rises on weekday mornings, falls at night and on weekends, drops during breaks and holidays, and peaks around exams. If we can predict these cycles a day ahead, we can switch servers off before demand falls and back on before it rises, instead of reacting late or never reacting at all.

A second opportunity comes from the power grid. The carbon cost of a kilowatt-hour changes from hour to hour, depending on how much of the grid's power is coming from renewables. Work that doesn't need to run immediately, such as backups, report generation or model training, can be moved to cleaner hours at no cost to users.

AURA combines these two ideas into one system. As students, our goal was to go beyond a slide-deck architecture and build something that actually runs end to end: real data in, a trained model, a measurable result, and a working cloud deployment that faculty can open in a browser.

2. Problem Statement and Objectives

Problem. University data centers run static, over-provisioned infrastructure, so they waste energy whenever demand is low. Existing green-cloud research mostly targets generic enterprise or IoT workloads. It overlooks the predictable, calendar-driven demand of a campus: semester start, exam weeks, weekends, holidays and breaks.

Objectives.

1. Build an AI-driven resource allocation framework for university data-center workloads.
2. Predict academic-calendar demand cycles a day ahead so servers scale up and down before demand changes, not after.
3. Schedule deferrable work into low-carbon hours, using real grid carbon-intensity data.
4. Reduce energy use and carbon footprint, and measure the reduction honestly, against a clear baseline.
5. Keep the service level agreement (SLA): enough capacity for interactive users in at least 99.5% of hours.
6. Show live energy, carbon, cost and SLA figures on a secure web dashboard deployed on AWS.

Success criteria we set for ourselves.

Criterion	Target	Achieved
Forecast beats a naive baseline on unseen data	Lower MAE and RMSE	MAE −20%, RMSE −42%
Energy saving vs always-on	> 30%	39.5%
SLA compliance	≥ 99.5%	99.53%
Settings chosen without looking at test data	Tune on validation only	Done
Running cloud deployment	Public, authenticated URL	EC2 + S3 + Cognito + DynamoDB

3. Related Work and Project Contribution

Previous work falls into three groups. AURA borrows from each and focuses on the gap between them: university-specific demand.

Area	Typical approach	What it misses for a campus
Reactive autoscaling (e.g. AWS Auto Scaling, Kubernetes HPA)	Add or remove servers when CPU crosses a threshold	Always one step late; ignores the known academic calendar
Workload forecasting (ARIMA, Holt-Winters, LSTM)	Predict future load from past load	Usually evaluated on enterprise or web traces, not university cycles or holidays
Carbon-aware computing (e.g. Google's carbon-intelligent platform, Carbon Intensity API work)	Move flexible jobs to cleaner hours or regions	Rarely combined with demand forecasting in one policy

Our literature survey of 13 papers (docs/LitSurvey_and_ResearchGaps.docx) shaped these choices. Ilager and Buyya [7] note that computing and cooling are usually optimised separately and that ML models transfer poorly between facilities. Wang and Yang [8] pair LSTM forecasting with a DQN scheduler, but compare against weak, unnamed baselines; we therefore report against a named seasonal-naive baseline. Beena et al. [9] show that few frameworks use live carbon-intensity data, and Zhao, Zhou and Li [10] show that cost-only optimisation is not the same as low carbon. Manassra and Işık [11] report large campus savings, but only in simulation and without one unified framework, which is the gap AURA targets.

Our contribution.

- **Calendar-aware forecasting:** an LSTM with cyclic time features, Thai public-holiday flags and last week's load, trained on real university building data.
- **One combined policy:** forecast-driven server provisioning plus carbon-aware placement of batch work, with the contribution of each measured separately.
- **Honest evaluation:** every setting is tuned on a validation period and reported once on an unseen test period. When an early model lost to the baseline, we replaced it and reported that too.
- **A working product:** an interactive dashboard and API that let a user run their own what-if scenarios or upload their own load data, deployed on AWS.

4. System Architecture

AURA is a pipeline of four layers: data comes in, the model forecasts demand, the scheduler decides how many servers to run and when to run batch work, and the dashboard shows the result.

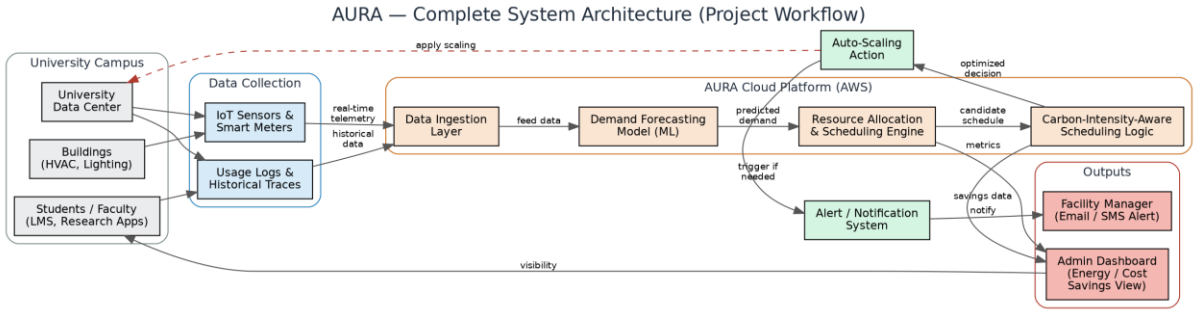


Figure 1. AURA system architecture.

Layer	What it does	Implemented with
Data	Building load, calendar and grid carbon intensity	CU-BEMS dataset, Carbon Intensity API, preprocess.py
Forecasting	Next-24-hour demand from the past 7 days	TensorFlow LSTM (train_lstm.py, engine_server.py)
Scheduling	Servers to keep on each hour; where to place batch work	simulate.py allocation policy
Application	API, auth, storage and dashboard	Express API, Cognito, DynamoDB, React dashboard on S3

Request flow in the deployed system. The browser loads the dashboard from S3 and the user signs in with Cognito. The dashboard calls the API on EC2 with the user's token. The API checks the token, reads results from DynamoDB through an IAM role, and calls the Python forecasting engine for live simulations and CSV uploads.

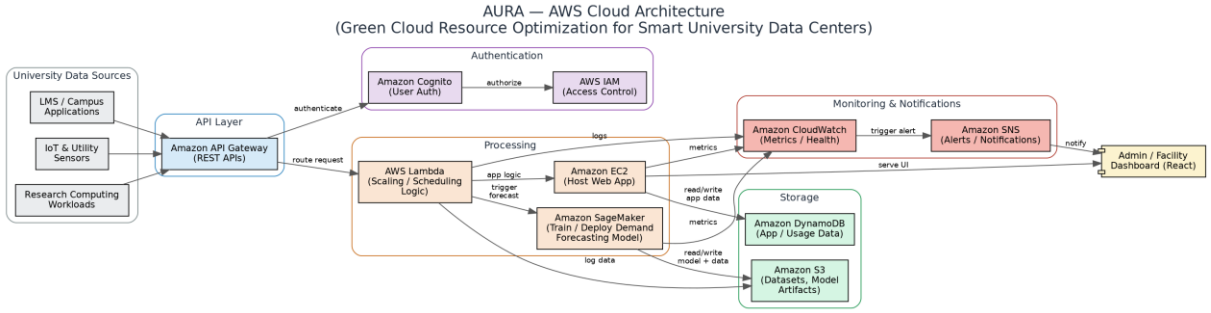


Figure 2. Planned AWS architecture. Section 9 explains which parts we deployed and which are future work.

5. Dataset and Preprocessing

We used real metered electricity data from a university building, not synthetic traces, so the model learns genuine academic cycles.

Property	Value
Dataset	CU-BEMS, Chulalongkorn University, Bangkok (Pipattanasomporn et al., Scientific Data 7, 241, 2020)
Licence	CC BY 4.0
Size	14 CSV files, 788 MB, 5,432,489 rows at 1-minute resolution
Period	1 July 2018 to 31 December 2019, 7 floors
Content	Air-conditioning, lighting and plug-load power (kW); temperature, humidity, light
Carbon data	Hourly UK grid carbon intensity (gCO_2/kWh) from the Carbon Intensity API, 3,481 hours

Why the UK grid? Thailand has no free public hourly carbon-intensity feed. The UK API gives a real, varying signal, so the scheduler is tested on genuine clean and dirty hours. The method works with any grid's data.

Preprocessing steps (src/ml_model/preprocess.py).

7. Parse timestamps, drop unparseable and duplicate rows, and join 2018 and 2019 for each floor.
8. Sum each floor's meters into total, AC, lighting and plug kW, and average the environment sensors.

9. Resample to 15-minute means and interpolate gaps of up to 1 hour; longer gaps stay empty.
10. Build the building total only from time steps where all 7 floors have data, so a missing floor never looks like low demand. This kept 41,766 of 52,704 steps.
11. Add calendar features: hour, weekday, month, weekend, and Thai public holidays.

Data split (chronological, no shuffling).

Split	Period	Used for
Train	1 Jul 2018 – 31 May 2019	Fitting the LSTM
Validation	1 Jun 2019 – 11 Aug 2019	Choosing model size, lookback and scheduler settings
Test	12 Aug 2019 – 21 Oct 2019 (1,704 h)	Reported once, at the end

Data issues we found and handled: floor 3 has a roughly 40-day gap in power data; 2018 sensor columns are 23–39% empty; the 2019 floor-6 file stops on 22 October 2019. These are all documented in dataset/processed/preprocess_report.txt.

6. Demand Forecasting Model

On the unseen test period, the LSTM's average error was 35.9 kW, 20% lower than the same-hour-last-week baseline; its RMSE was 48.0 kW, 42% lower.

Model design (src/ml_model/train_lstm.py).

- **Input:** the past 7 days of hourly load (168 steps), plus cyclic hour and weekday features, Thai public-holiday flags, and the load from the same hour one week earlier.
- **Output:** the next 24 hourly load values, forecast once a day at local midnight.
- **Network:** LSTM with 128 units and dropout 0.2, trained for up to 60 epochs with batch size 64.
- **Baseline:** seasonal naive, which predicts each hour as the same hour last week. It is simple but strong on weekly-cycle data, so beating it is meaningful.

Test-period results (12 Aug – 21 Oct 2019).

Model	MAE (kW)	RMSE (kW)	MAPE
AURA LSTM	35.9	48.0	20.5%
Same hour last week	44.9	83.3	24.0%

How we chose the settings (validation MAE). 128 units scored 38.0 kW against 38.2 kW for 32 units and 39.5 kW for 64; a 7-day lookback beat a 3-day lookback (39.2 kW). Everything was chosen on validation data only.

What we learned. Our first model, without holiday and future-calendar features, scored a test MAE of 46.8 kW, which is worse than the baseline. Adding the holiday flags and last week's load fixed this. The much lower RMSE shows that the LSTM avoids the large misses the baseline makes when a holiday breaks the weekly pattern.

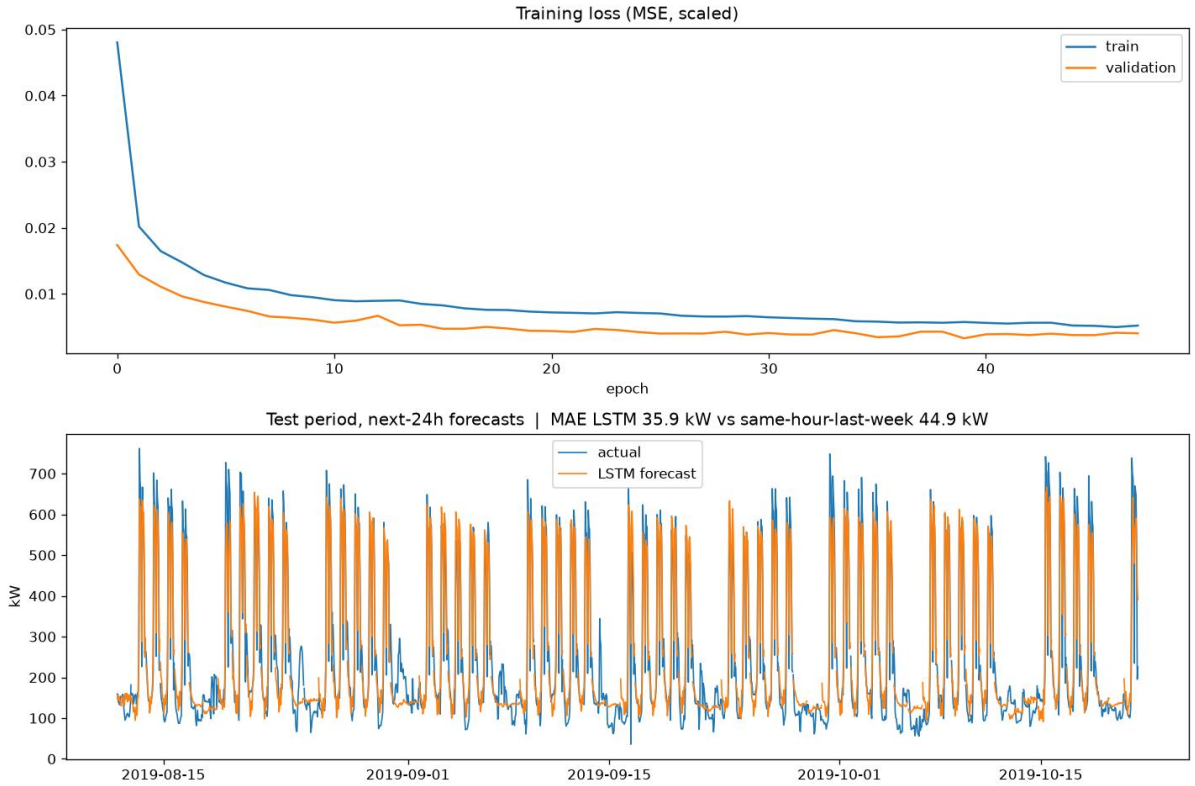


Figure 3. LSTM training history and day-ahead forecasts against actual load on the test period.

7. Forecast-Driven, Carbon-Aware Allocation Simulation

The scheduler replays the 1,704-hour test period hour by hour, using only forecasts made from earlier data, and compares AURA with a static data center that keeps all 48 servers on 24/7.

AURA's policy, every day at midnight:

12. Forecast the next 24 hours of demand with the LSTM.
13. For each hour, plan ceil(forecast × (1 + headroom) + buffer) servers, never fewer than 4.
14. Take the day's deferrable batch work (20% of the load) and place it in the hours with the lowest grid carbon intensity that still have spare capacity.
15. At run time, serve interactive users first. Batch work that doesn't fit moves to the next hour. If real demand exceeds the planned servers, scale up reactively and count that hour as an SLA miss.

Choosing the safety margin. We tested headroom and buffer combinations on the validation period with one rule: the highest energy saving with SLA ≥ 99.5%. This picked 25% headroom + 4 buffer servers, which was then frozen before the test run.

Validation setting	Energy saved	SLA
15% + 2 servers	42.5%	97.24%
15% + 4	39.7%	99.46%
25% + 4 (selected)	38.0%	99.87%
25% + 6	35.2%	99.93%
50% + 8	28.4%	100%

Modelling assumptions (stand-ins, not measurements): building load scaled to a 48-server fleet of c5.xlarge-class machines; 0.12 kW idle and 0.30 kW at full load per server; cooling overhead (PUE) 1.5; electricity at $0.12/kWh.

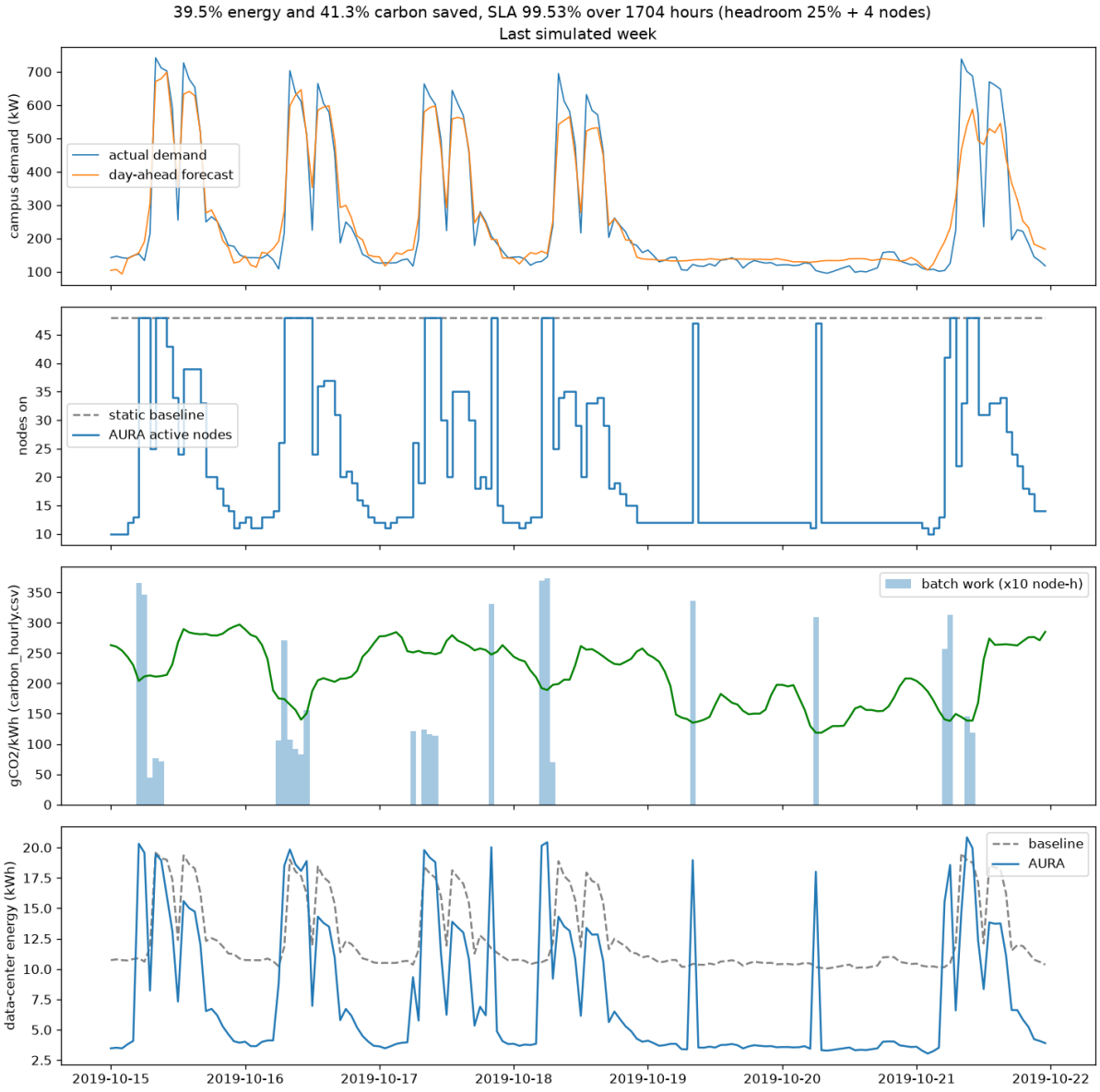


Figure 4. The last simulated week: AURA keeps far fewer servers on than the static baseline and shifts batch work into low-carbon hours.

8. Application Implementation

AURA is a three-part application: a React dashboard, a Node.js/Express API, and a Python forecasting engine that the API keeps warm in memory.

Part	Technology	Key responsibilities
Dashboard	React 19, Vite, Tailwind CSS, Recharts, AWS Amplify	Seven pages, charts, sign-in, dark mode, live refresh, Ctrl+K command palette
API	Node.js, Express	REST endpoints, Cognito token checks, file or DynamoDB storage,

		CSV upload validation
Engine	Python, TensorFlow, pandas	Runs the LSTM and the allocation policy on demand (engine_server.py)

Dashboard pages. Overview (headline savings and charts), Forecast (next 24 hours), Allocation (per-server state with a live fleet replay), Carbon and Energy, Alerts, Architecture, and Simulator.

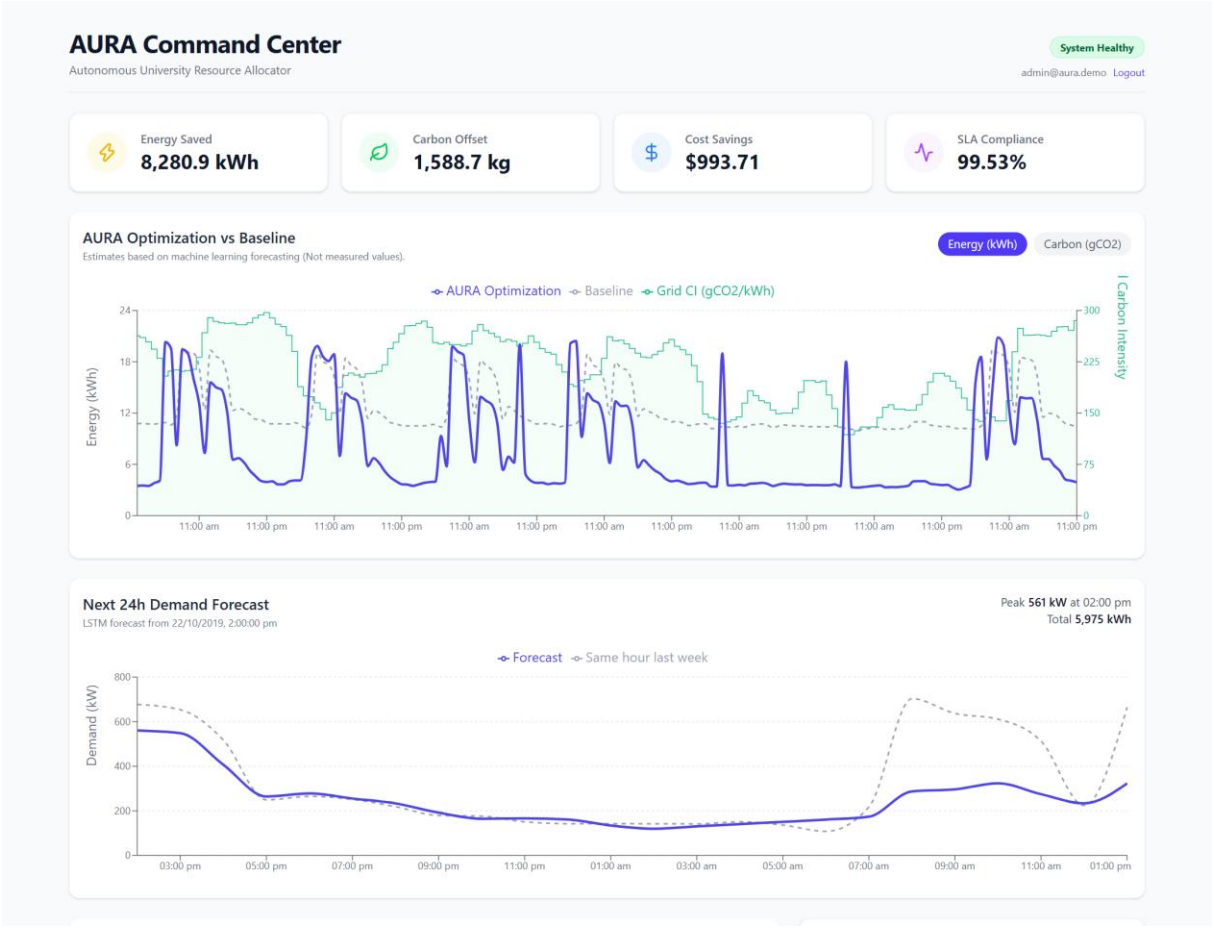


Figure 5. The Overview page: energy saved, carbon offset, cost saving and SLA, with AURA vs the always-on baseline.

Interactive features on the Simulator page.

- **What-if simulation:** choose dates, server count, server power, deferrable share and grid profile; the live LSTM re-runs the policy (POST /api/simulate).
- **Use your own data:** upload a timestamp,kw CSV of 16–31 days; the first 14 days are history and the rest is simulated (POST /api/upload).
- **Talk to AURA:** type a request in plain English, for example "exam week with 30 servers on a dirty grid".
- **Optimizer, Carbon Advisor, Anomaly detection, AI Explainer, and report export** (CSV and a printable A4 report).

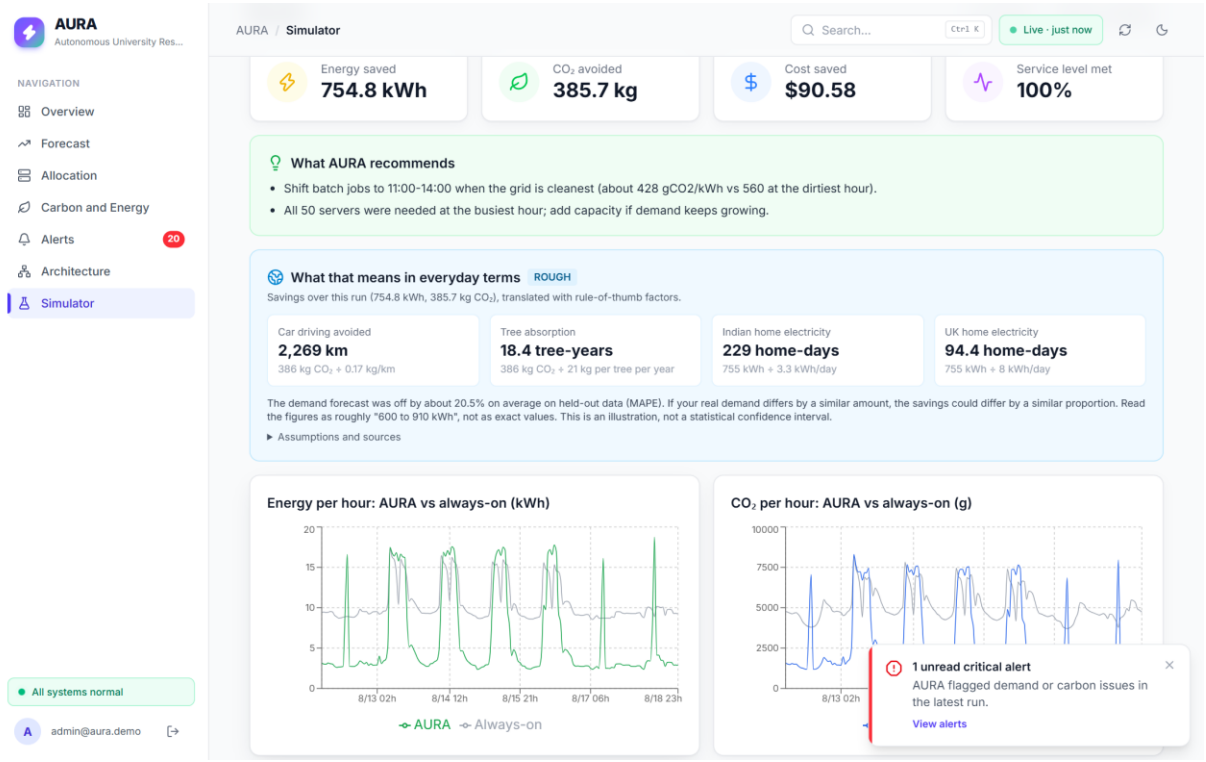


Figure 6. Simulator results for 50 servers, 12–18 Aug 2019: 754.8 kWh and 385.7 kg CO_2 saved, 100% service level, with recommendations and everyday equivalents.

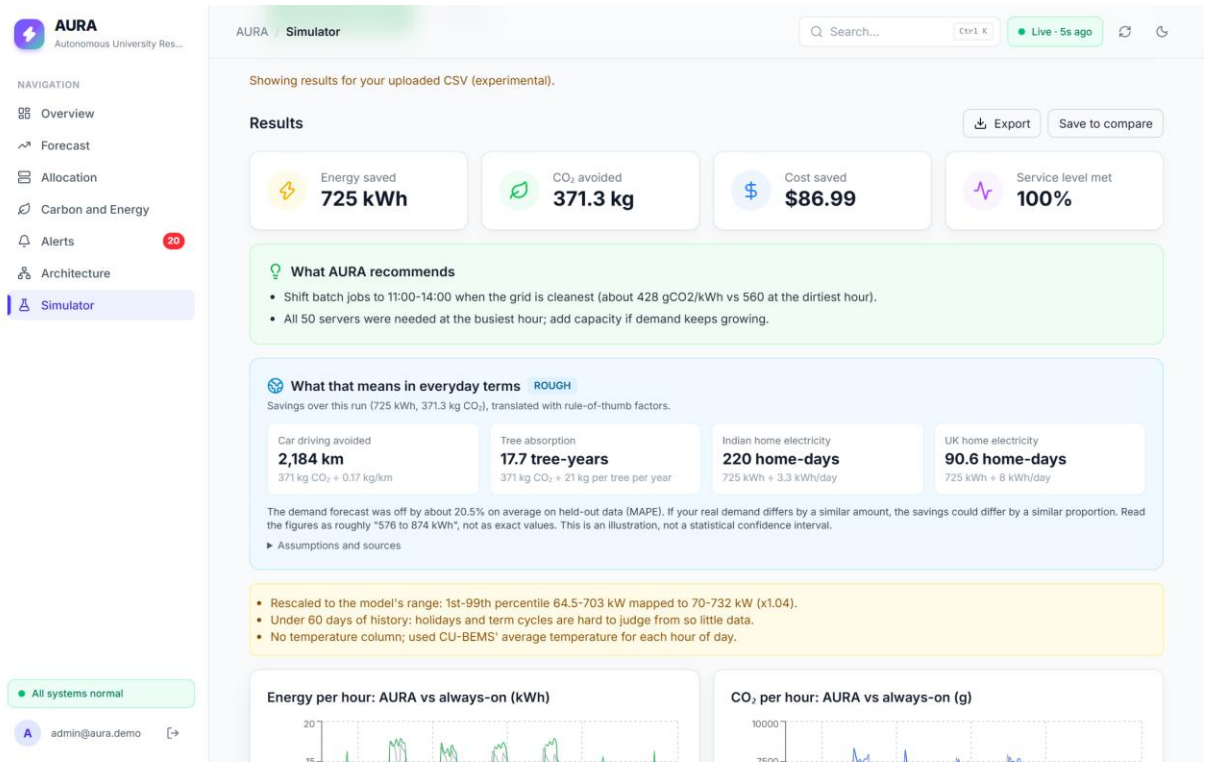


Figure 7. Results after uploading demo/demo_input.csv (real CU-BEMS data, 14–20 Oct 2019): 725 kWh and 371.3 kg CO_2 saved, $86.99, 100% service level. The yellow box lists the model's data-quality warnings.

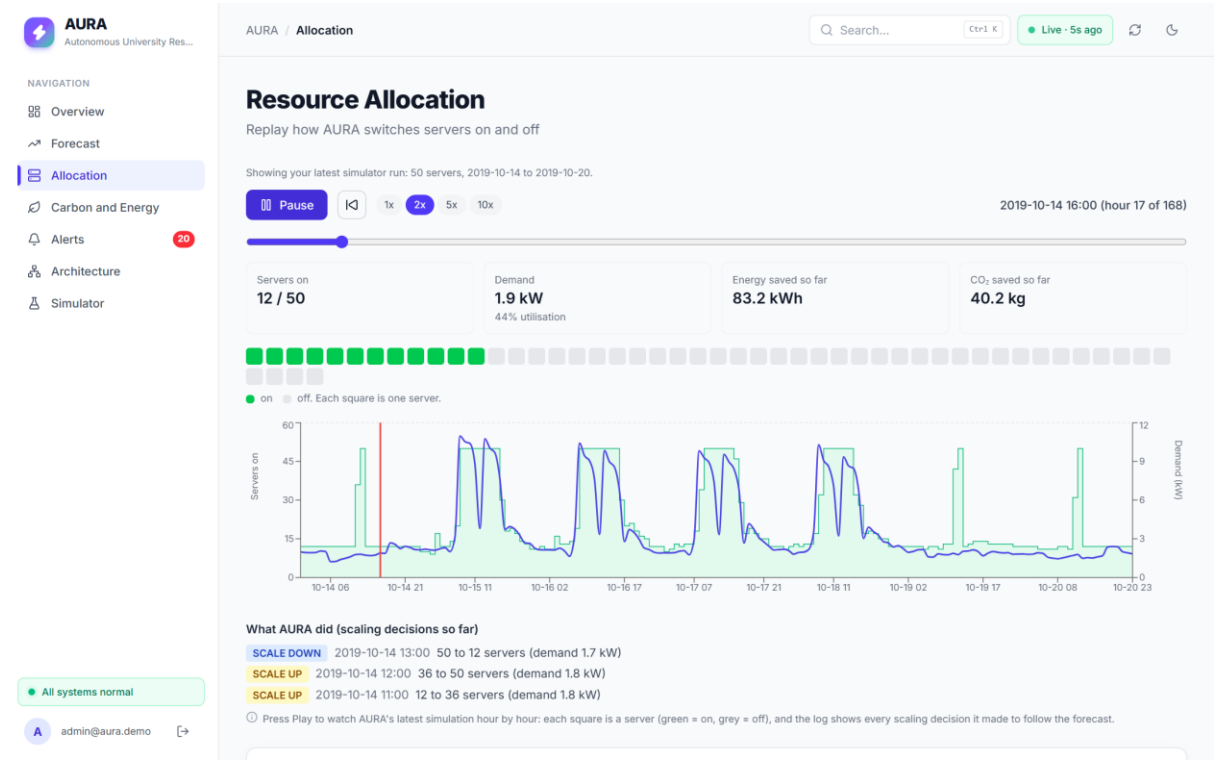


Figure 8. Fleet replay on the Allocation page: each square is a server; the log shows AURA scaling from 50 down to 12 servers as demand falls.

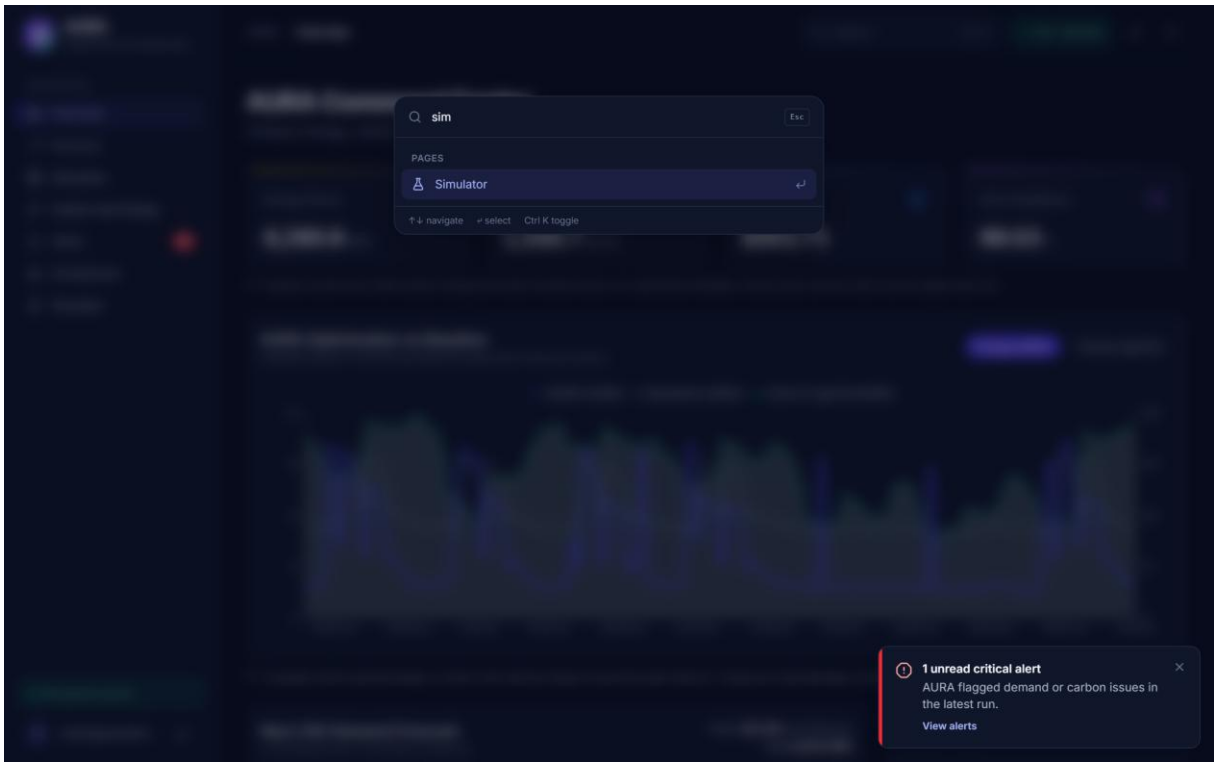


Figure 9. Dark mode with the Ctrl+K command palette open.

Team contributions. Agrani built the data pipeline, the LSTM and the simulation. Prakul built the backend API and storage layer, the dashboard redesign (dark mode, live updates, command palette), the CSV-upload and login fixes, and the AWS deployment. Pranav built the frontend and the interactive features: simulator, optimizer, Talk to AURA, explainer, advisor, anomalies and export.

9. AWS Deployment Design

AURA runs live on AWS in the Mumbai region (ap-south-1), entirely within the AWS free plan: the dashboard on S3, sign-in through Cognito, data in DynamoDB, and the API with the ML model on one EC2 server.

Service	Resource	Role in AURA
Amazon S3	aura-dashboard-<account> static website	Hosts the built React dashboard
Amazon Cognito	User pool aura-users, app client aura-dashboard	Dashboard sign-in; admin-created users only, 10+ character passwords
Amazon DynamoDB	Table AURA, on-demand, keys type + id	720 hourly energy rows, savings, 50 server allocations, 20 alerts, forecast
Amazon EC2	c7i-flex.large (2 vCPU, 4 GB), Amazon Linux 2023	Express API behind nginx, plus the TensorFlow engine; restarts automatically through systemd
AWS IAM	Role aura-ec2-role	Lets EC2 read only the AURA table, with no keys stored on the server
Security group	aura-sg	Port 80 open to the public; SSH only from the team's IP

Security decisions. Every /api/* route needs a valid Cognito token, and requests without one get HTTP 401. The server reads DynamoDB through a least-privilege IAM role, not stored access keys. Cross-origin requests are accepted only from the dashboard's S3 address.

Live resource check (read from the AWS account with the AWS CLI on 22 October 2026):

Resource	Verified state
EC2 instance i-0121f0e42dd2bf6e4	running, c7i-flex.large, ap-south-1b, public IP 3.110.171.176, instance profile aura-ec2-profile
DynamoDB table AURA	ACTIVE, PAY_PER_REQUEST, keys type + id, 792 items
Cognito user pool aura-users (ap-south-1_V8VsDhjm3)	1 user, minimum password length 10, admin-created users only
S3 bucket aura-dashboard-695071477941	static website, index document index.html, 6 objects (1.29 MB)

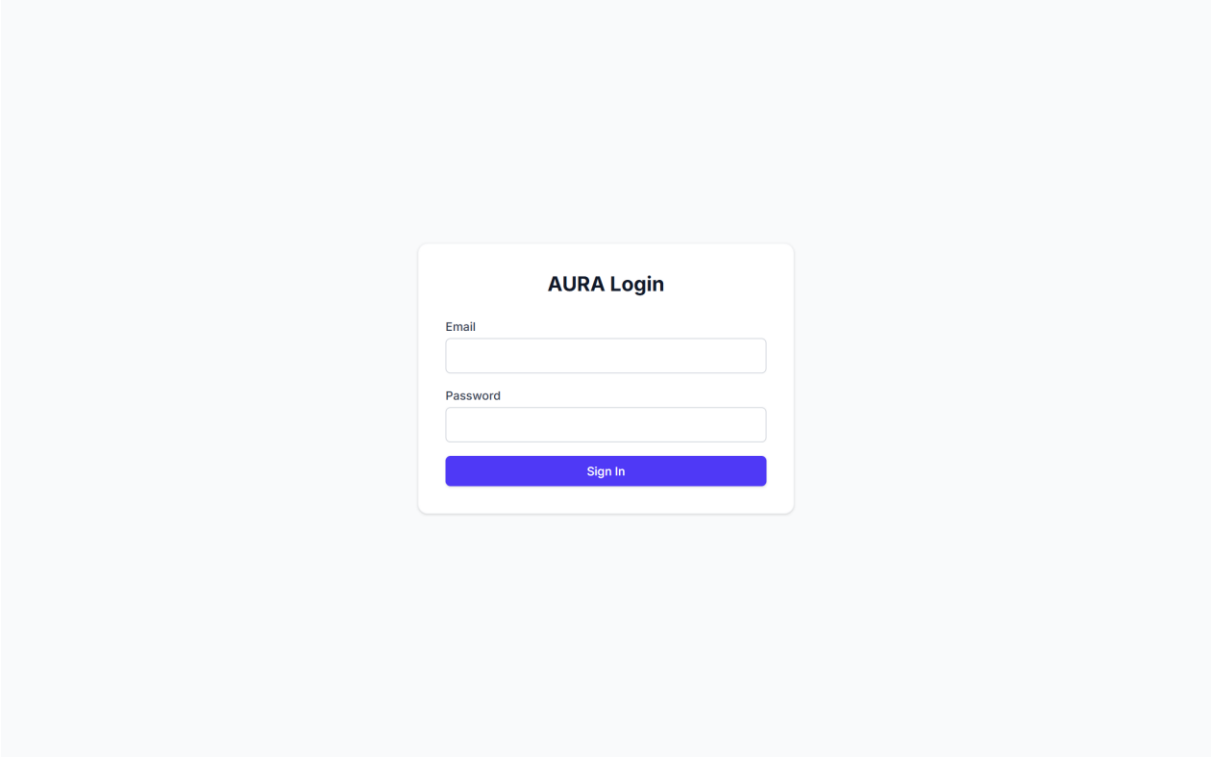


Figure 10. The Cognito sign-in page, served live from the S3 website.

Designed but not yet deployed. The full design in Figure 2 also includes SNS email alerts with an alert-publisher Lambda, CloudWatch alarms, API Gateway, and SageMaker for scheduled retraining. Agrani's CloudFormation template (src/aws/aura-stack.yaml) already defines SNS, the Lambda and the alarm.

A bug found during deployment. When we first switched to DynamoDB, the dashboard showed "API unreachable". Each alert's own type field (such as "scaling") was overwriting the table's partition key, so alerts were stored under the wrong key and none could be found. We fixed store.js to keep the two fields separate, cleaned the table and re-loaded the data. This bug never appeared in local testing, because local runs read JSON files rather than DynamoDB.

10. Testing and Quality Assurance

All 87 backend tests pass, and the system was also checked end to end on the live AWS deployment.

Level	What is tested	Where
Unit and API tests (87 passing)	Every endpoint, input validation, the file and DynamoDB stores, optimizer, advisor, anomaly detection, Talk to AURA parsing	src/backend/test/, src/backend/tests/ (Node test runner)
Integration tests	Different simulator inputs produce different outputs; report export	src/frontend/tests/
Model evaluation	Validation-only tuning; one final test-period run; comparison with the seasonal-naive baseline	results/model/metrics.json
End-to-end on AWS	Dashboard loads from S3; API health; 401 without a token; upload of a real CSV through the live model	Manual checks, 22 Oct 2026

Demo input built from real data. For the upload feature we built demo/demo_input.csv from real CU-BEMS readings, with nothing invented: 504 hourly rows from 30 September to 20 October 2019, each the mean of four complete 15-minute readings. The simulated week includes a Thai public holiday (Monday 14 October, King Bhumibol Memorial Day in lieu). Running it gives 40.5% energy saved, 42.6% CO_2 saved and 100% SLA.

Latest test run (22 Oct 2026): 87 tests, 87 passed, 0 failed, in 0.95 s (npm test in src/backend).

Bugs found and fixed by testing.

Bug	Cause	Fix
Chart hover dot stuck on the first day	X-axis keyed on time of day, which repeats each day	Key the axis on the full timestamp
Upload looked like it did nothing	Results appeared at the top of a long page	Confirmation message and auto-scroll to the results
"Already a signed in user" on reload	Sign-in state kept only in memory	Restore the existing Cognito session on load
"API unreachable" on AWS	Alert type overwrote the DynamoDB key	Separate the fields; re-load the table
Run simulation did nothing with default inputs	Peak-power field (min 11, step 5) rejected its own default of 250 W	Set the minimum to 15 so 250 is a valid step

11. Results and Discussion

Over the 71-day test period, AURA used 39.5% less energy and emitted 41.3% less CO_2 than always-on provisioning, and had enough capacity for interactive users in 99.53% of hours.

Measure (test period, 1,704 h)	AURA	Static baseline	Change
Data-center energy	12,674 kWh	20,955 kWh	−39.5% (8,281 kWh)
Emissions	2,256 kg CO_2	3,845 kg CO_2	−41.3% (1,589 kg)
Electricity cost	–	–	−$993.71
Average servers on	21.0	48	−56%
SLA compliance	99.53%	100%	−0.47 points

Where the savings come from. Almost all of the energy saving comes from forecast-driven provisioning: running 21 servers on average instead of 48. Carbon-aware placement of batch work adds 2.3 percentage points of carbon reduction on top (41.3% vs 39.0% without it, about 88 kg CO_2). It costs 0.4 points of SLA, because batch capacity spread across busy hours no longer doubles as spare headroom.

Discussion.

- **The trade-off is tunable.** The validation grid in Section 7 shows that more headroom buys a higher SLA at the cost of savings. A campus could choose 100% SLA at 28% savings, or accept 97% SLA for 42% savings.
- **Forecasting matters most around holidays.** The baseline's RMSE is almost double the LSTM's, because the baseline copies last week's pattern straight into a holiday.
- **Results generalise only partly.** On our real-data demo week, AURA saved 40.5% energy, consistent with the main result. Data from other buildings would need retraining.

- **What surprised us:** the carbon gain from shifting batch work was smaller than we expected. Most of the green benefit comes simply from switching idle servers off.

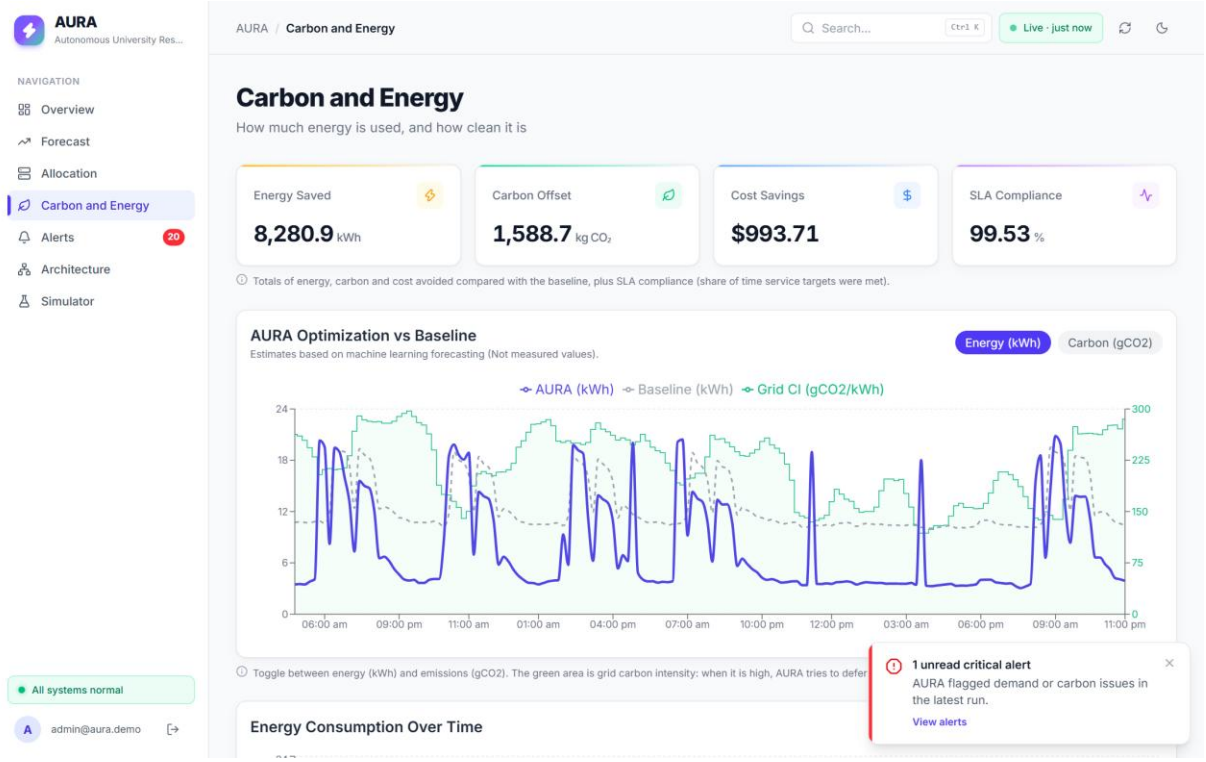


Figure 11. Carbon and Energy page: totals over the test period and AURA's hourly energy against the baseline and grid carbon intensity.

12. Limitations and Future Work

The biggest limitation is that building electricity load stands in for server demand. The method is sound, but the absolute numbers are a simulation, not measurements from a real data center.

Limitations.

- **Proxy demand:** CU-BEMS building load is scaled to a 48-server fleet; server power, PUE and price are assumptions.
- **Mismatched grid:** carbon intensity comes from the UK grid while the load comes from Bangkok, because no free Thai hourly feed exists.
- **One building:** the model is trained on one site, so uploaded data from elsewhere gives indicative results only.
- **Deployment shortcuts:** the demo uses plain HTTP, a single EC2 server, and an IP address that changes if the server restarts.

Future work.

16. Collect real server metrics (CPU, requests) from a campus data center and retrain on them.
17. Use the local grid's carbon data (for example India's grid for an Indian campus).
18. Add HTTPS with CloudFront and a custom domain, and put the API behind API Gateway.
19. Deploy SNS email alerts, CloudWatch alarms and scheduled SageMaker retraining from the CloudFormation design.
20. Let the policy act on real infrastructure through EC2 Auto Scaling, not just simulate it.

13. Conclusion

AURA shows that a university data center can cut its energy use by about 40% while still serving users in more than 99.5% of hours, simply by predicting the academic calendar a day ahead.

We built the whole chain ourselves: cleaning 5.4 million rows of real university data, training an LSTM that beats a strong baseline on unseen data, designing a carbon-aware scheduler, and wrapping it in a dashboard deployed on AWS with real authentication and a cloud database. Along the way we learned that honest evaluation matters as much as the model. Our first model lost to the baseline, and tuning only on validation data made our final numbers trustworthy. Deploying to the cloud also exposed a storage bug that local testing never could.

The approach is general. With real server metrics and local carbon data, the same pipeline could run on any campus.

Acknowledgements

We thank Dr. Priya V for guidance throughout the course, the CU-BEMS team at Chulalongkorn University for publishing their building data openly, and National Grid ESO for the Carbon Intensity API.

References

21. M. Pipattanasomporn et al., "CU-BEMS, smart building electricity consumption and indoor environmental sensor datasets," Scientific Data, vol. 7, 241, 2020. doi:10.1038/s41597-020-00582-3; data: doi:10.6084/m9.figshare.11726517
22. National Grid ESO, Carbon Intensity API. https://carbonintensity.org.uk
23. S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
24. A. Radovanović et al., "Carbon-aware computing for datacenters," IEEE Transactions on Power Systems, vol. 38, no. 2, 2023.
25. R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed., OTexts, 2021.
26. Amazon Web Services, documentation for Amazon EC2, S3, Cognito, DynamoDB and IAM. https://docs.aws.amazon.com
27. S. Ilager and R. Buyya, energy and thermal management of data centers using machine learning: a taxonomy (team literature survey, paper 2).
28. Wang and Yang, cloud resource allocation via machine learning: LSTM forecasting with a DQN scheduler (team literature survey, paper 12).
29. Beena et al., carbon-aware task scheduling on AWS/Kubernetes using real-time carbon-intensity APIs (team literature survey, paper 7).
30. Zhao, Zhou and Li, CFWS: deep-reinforcement-learning workload shifting for green data centers (team literature survey, paper 9).
31. Manassra and Işık, AI-based smart campus energy management (team literature survey, paper 6).

Appendix A. Running the Project

Live (AWS): open the S3 dashboard URL and sign in with a Cognito user created by an admin.

Live dashboard: http://aura-dashboard-695071477941.s3-website.ap-south-1.amazonaws.com (API: http://3.110.171.176)

Locally (Windows, Node 18+ and Python 3.11):

32. Clone the repository: git clone https://github.com/agranianupam/AURA_Cloud_Project_2026.git
33. Python engine: python -m venv src/ml_model/.venv, then src/ml_model/.venv/Scripts/pip install -r src/ml_model/requirements.txt
34. Backend: in src/backend, run npm install, then start it with AUTH_DISABLED=true and PYTHON pointing at the venv's Python, using npm start (port 3000).
35. Frontend: in src/frontend, create .env.local (Appendix B), then run npm install and npm run dev (port 5173).
36. Open http://localhost:5173. To try the upload, use demo/demo_input.csv on the Simulator page.

Regenerate results: python src/ml_model/preprocess.py, train_lstm.py, then simulate.py. **Tests:** npm test in src/backend.

Appendix B. Configuration Reference

Backend (environment variables).

Variable	Default	Meaning
PORT	3000	API port
DYNAMODB_TABLE	unset	If set, read data from this DynamoDB table; otherwise from results/simulation/*.json
AWS_REGION	us-east-1	Region of the table (ap-south-1 in our deployment)
COGNITO_USER_POOL_ID, COGNITO_CLIENT_ID	unset	Cognito pool used to verify sign-in tokens
AUTH_DISABLED	unset	true turns off token checks (local demos only)
CORS_ORIGIN		Allowed dashboard origin
PYTHON	auto-detected	Python used for the forecasting engine

Frontend (src/frontend/.env.local, read at build time).

Variable	Meaning
VITE_API_BASE_URL	API address, e.g. http://localhost:3000
VITE_USE_MOCK	true shows built-in mock data with no backend
VITE_AUTH_DISABLED	true skips the sign-in page
VITE_COGNITO_USER_POOL_ID, VITE_COGNITO_CLIENT_ID	Cognito pool for sign-in

Simulation defaults: 48 servers (minimum 4), 0.12 kW idle and 0.30 kW peak, PUE 1.5, 25% headroom + 4 buffer servers, 20% deferrable work, $0.12/kWh.

Upload format: CSV with header timestamp,kw, 16–31 days, at most 5 MB.

Appendix C. Glossary

Term	Meaning
Carbon intensity	Grams of CO_2 emitted per kWh of electricity at a given hour
Deferrable (batch) work	Jobs that can wait a few hours, such as backups or reports
Headroom	Extra capacity planned above the forecast, as a percentage
LSTM	Long short-term memory, a neural network for sequences
MAE / RMSE / MAPE	Mean absolute error / root mean squared error / mean absolute percentage error
PUE	Power usage effectiveness: total facility power divided by IT power
Seasonal naive	Baseline forecast that repeats the same hour from last week
SLA	Service level agreement; here, the share of hours with enough capacity for interactive users
Static provisioning	Keeping all servers on all the time (our baseline)
Cognito / DynamoDB / EC2 / IAM / S3	AWS services for sign-in / NoSQL database / virtual servers / permissions / file and website storage